Assignment 1:

Map Coloring as a Constraint Satisfaction Problem

COMP 380: Introduction to Artificial Intelligence | Fall 2026 Semester

University of the Fraser Valley

Ryan Austin [300211146]

Instructor: Amir Shabani

October 24, 2026

GitHub Repository:
https://github.com/DarksteelCitadel/COMP380-Assignment1

1. Abstract

This report describes a Python program that colors the 13 provinces and territories of Canada so that no two neighboring regions share a color. The problem is formulated as a constraint satisfaction problem (CSP), with one variable per region, a domain of user-supplied colors, and a not-equal constraint for each pair of neighbors. The solver uses recursive backtracking search with Minimum Remaining Values (MRV) variable ordering, Least Constraining Value (LCV) value ordering, and forward checking. The program reads the color list from the user, or uses Red, Green, and Blue by default, and prints each region with its assigned color. Testing with the default colors produced a valid three-color map. Testing with two colors correctly reported that no coloring exists. The results show that the CSP formulation and backtracking search solve this problem directly, and they illustrate how pruning and ordering heuristics reduce the search.

2. Introduction

Map coloring is a standard example of a constraint satisfaction problem. Each region must be assigned a color, and the only constraint is that neighboring regions must receive different colors. Although the rule is simple, the problem illustrates the main ideas of CSPs: variables, domains, constraints, and systematic search with pruning (Poole & Mackworth, 2017; Russell & Norvig, 2010).

2.1 Problem Definition

The problem is formulated as a CSP with three components:

Variables: The 13 provinces and territories, identified by their two-letter postal codes (AB, BC, MB, NB, NL, NS, NT, NU, ON, PE, QC, SK, YT).

Domains: The list of colors supplied by the user. The default domain is {Red, Green, Blue}.

Constraints: For every pair of neighboring regions X and Y, color(X) ≠ color(Y). These are binary, not-equal constraints.

The goal is to find any assignment of a color to every variable that satisfies all constraints.

Modeling decisions: The adjacency graph was built from the map of provinces and territories on Wikipedia (Wikipedia, 2026). Two borders lie across water: Nunavut-Ontario and Nunavut-Quebec, which meet in Hudson Bay and James Bay. Manitoba-Nunavut is a land border. Prince Edward Island is an island with no land border, so it is joined to New Brunswick

and Nova Scotia across the Northumberland Strait. Without these links, PE would form a separate component, and the graph would not be connected as the assignment requires.

3. Experimental Setup

3.1 Methodology and Algorithm

The solver is a recursive backtracking search over partial assignments. At each step it does the following:

1. Select a variable (MRV). Choose the unassigned region with the fewest legal colors remaining. Ties are broken by choosing the region with the most neighbors (degree heuristic). This tends to detect dead ends early.
2. Order the values (LCV). Try colors in the order that rules out the fewest options for neighboring regions first. Ties keep the user's original color order.
3. Apply forward checking. After assigning a color to a region, remove that color from the domains of its unassigned neighbors. If any neighbor's domain becomes empty, reject this color immediately.
4. Recurse or backtrack. If the assignment is complete, return it. Otherwise, recurse. If the recursive call fails, undo the assignment and try the next color.

Pseudo-code:

```
FUNCTION COLOR_MAP(colors):
    FOR each region v IN REGIONS DO
        domains[v] = set(colors)
    END FOR
    RETURN BACKTRACK({}, domains, colors)
END FUNCTION

FUNCTION BACKTRACK(assignment, domains, colors):
    IF size(assignment) = number of regions THEN
        RETURN assignment
    END IF

    var = SELECT_UNASSIGNED_VARIABLE(assignment, domains)
        // MRV: fewest remaining colors; ties broken by most neighbors

    FOR each color IN ORDER_DOMAIN_VALUES(var, assignment, domains, colors) DO
        new_domains = copy of domains
        dead_end = FALSE

        // Forward checking
        FOR each neighbor IN NEIGHBORS(var) DO
```

```
            IF neighbor is NOT in assignment THEN
                remove color from new_domains[neighbor]
                IF new_domains[neighbor] is empty THEN
                    dead_end = TRUE
                    BREAK
                END IF
            END IF
        END FOR

        IF dead_end THEN
            CONTINUE
        END IF

        assignment[var] = color
        result = BACKTRACK(assignment, new_domains, colors)
        IF result != None THEN
            RETURN result
        END IF
        remove var from assignment       // undo the choice
    END FOR

    RETURN None
END FUNCTION

FUNCTION ORDER_DOMAIN_VALUES(var, assignment, domains, colors):
    legal = [color IN colors WHERE color IS IN domains[var]]
    RETURN legal sorted by number of unassigned neighbors
        that still have this color available (fewest first)
END FUNCTION
```

Complexity: In the worst case, backtracking search takes time exponential in the number of regions. For 13 regions with three colors, the search space has at most 3^{13}, or approximately 1.6 million assignments. MRV, LCV, and forward checking prune most of this space in practice. The solver does not implement the stronger arc consistency (AC-3) algorithm, which would detect some failures earlier. This is a possible extension.

3.2 Graph Model and Data

The adjacency graph is stored as a dictionary that maps each region to a list of its neighbors. The build_graph function converts the edge list into this structure, adding each edge in both directions so the graph is undirected. The graph has 13 vertices and 19 edges.

3.3 Input/Output and User Interaction

When the program starts, it asks the user for a comma-separated list of colors. Pressing Enter selects the default colors, Red, Green, and Blue. The program rejects duplicate colors and reports the error.

After solving, the program prints a table with one row per region. Each row shows the postal code, the full name, and the assigned color. If no valid coloring exists with the given colors, the program prints a message suggesting more colors.

4. Results and Discussion

4.1 Test Case 1: Default Colors (Red, Green, Blue)

Input: Enter (no colors typed)

```
=== Canada Map Coloring (CSP) ===

Enter colors separated by commas (press Enter for Red,Green,Blue):

Code   Province / Territory          Color
--------------------------------------------------
AB     Alberta                      Green
BC     British Columbia             Blue
MB     Manitoba                     Red
NB     New Brunswick                Green
NL     Newfoundland and Labrador    Green
NS     Nova Scotia                  Red
NT     Northwest Territories        Red
NU     Nunavut                      Green
ON     Ontario                      Blue
PE     Prince Edward Island         Blue
QC     Quebec                       Red
SK     Saskatchewan                 Blue
YT     Yukon                        Green

All neighboring regions have different colors.
PS C:\Users\Ryan Austin\Documents\GitHub\COMP380-Assignment1>
```

I checked all 19 edges against this output, and every pair of neighboring regions has different colors. For example, AB (Green) and BC (Blue) differ, and ON (Blue) and QC (Red) differ.

4.2 Test Case 2: Custom Colors (Crimson, Teal, Gold)

Input: Crimson, Teal, Gold

```
=== Canada Map Coloring (CSP) ===

Enter colors separated by commas (press Enter for Red,Green,Blue): Crimson,Teal,Gold

Code   Province / Territory          Color
--------------------------------------------------
AB     Alberta                      Teal
BC     British Columbia             Gold
MB     Manitoba                     Crimson
NB     New Brunswick                Teal
NL     Newfoundland and Labrador    Teal
NS     Nova Scotia                  Crimson
NT     Northwest Territories        Crimson
NU     Nunavut                      Teal
ON     Ontario                      Gold
PE     Prince Edward Island         Gold
QC     Quebec                       Crimson
SK     Saskatchewan                 Gold
YT     Yukon                        Teal

All neighboring regions have different colors.
PS C:\Users\Ryan Austin\Documents\GitHub\COMP380-Assignment1>
```

This test shows that the color list is supplied by the user and that the solver works with any number of colors of at least three.

4.3 Test Case 3: Insufficient Colors

Input: Red, Green

```
=== Canada Map Coloring (CSP) ===

Enter colors separated by commas (press Enter for Red,Green,Blue): Red, Green

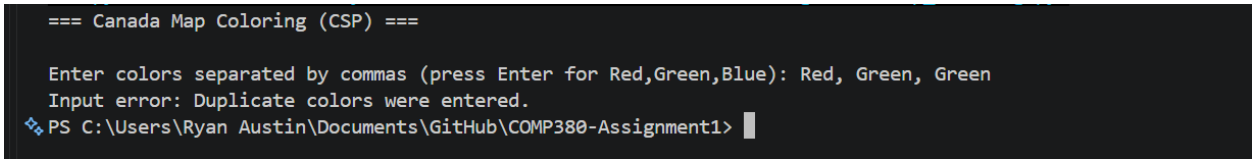
No valid coloring exists with 2 color(s). Try adding more colors (3 is enough for Canada).
PS C:\Users\Ryan Austin\Documents\GitHub\COMP380-Assignment1>
```

This test shows that the solver reports unsatisfiable instances. Canada's graph contains triangles, such as AB-BC-NT, so two colors cannot satisfy the constraints.

4.4 Test Case 4: Duplicate Colors

Input: Red, Green, Green

This test shows that the input validation rejects a color list containing repeated names before



the solver runs. Without this check, the duplicate would reduce the effective palette to two colors and the program would report an unsolvable instance, which would be misleading.

4.5 Discussion

The solver found a valid coloring for the default input. MRV selects the variable with the fewest remaining colors at each step, and LCV orders the colors so that neighbors keep the most options open. Forward checking removes impossible colors from unassigned neighbors, so dead ends are detected as soon as they arise.

The main limitation is that forward checking only looks one step ahead. Arc consistency would propagate constraints further, detecting failures earlier on larger or harder maps. The graph is also fixed in the code. Changing the map requires editing PROVINCES and EDGES.

5. Summary and Conclusions

This project formulated the map coloring problem for Canada as a CSP and solved it with backtracking search enhanced by MRV, LCV, and forward checking. The program works with any color list of at least three colors, and it correctly reports when a coloring is impossible.

The project shows how a problem can be modeled with variables, domains, and constraints, and how search heuristics reduce the work needed to find a solution. Modeling choices, such as treating water boundaries as adjacencies and connecting Prince Edward Island to the mainland, affect the result and should be stated clearly.

A future version could add a graphical display, which would make the colored map easier to read, and arc consistency (AC-3), which would make the search faster on harder instances.

6. References

Poole, D. L., & Mackworth, A. K. (2017). *Artificial intelligence: Foundations of computational agents* (2nd ed.). Cambridge University Press. https://artint.info

Russell, S., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed.). Pearson.

Wikipedia. (2026). *Provinces and territories of Canada*. Retrieved October 24, 2026, from https://en.wikipedia.org/wiki/Provinces_and_territories_of_Canada

7. Appendices

Appendix A: Adjacency Edge List

The 19 borders used in the graph, as defined in the EDGES list:

```
EDGES = [
    ("AB", "BC"), ("AB", "SK"), ("AB", "NT"),
    ("BC", "NT"), ("BC", "YT"),
    ("MB", "SK"), ("MB", "ON"), ("MB", "NU"),
    ("NB", "QC"), ("NB", "NS"), ("NB", "PE"),
    ("NL", "QC"),
    ("NS", "PE"),
    ("NT", "SK"), ("NT", "NU"), ("NT", "YT"),
    ("NU", "QC"), ("NU", "ON"),
    ("ON", "QC"),
]
```

Appendix B: Province and Territory Codes

```
PROVINCES = {
    "AB": "Alberta",
    "BC": "British Columbia",
    "MB": "Manitoba",
    "NB": "New Brunswick",
    "NL": "Newfoundland and Labrador",
    "NS": "Nova Scotia",
    "NT": "Northwest Territories",
    "NU": "Nunavut",
    "ON": "Ontario",
    "PE": "Prince Edward Island",
    "QC": "Quebec",
    "SK": "Saskatchewan",
    "YT": "Yukon",
}
```

Appendix C: Software Stack

- Python 3.14.3
- Standard library only: sys
- No external packages are required